Universidad Andrés Bello Online

Guía de Ejercicios
Unidad 2

Guía de Ejercicios Unidad 2

Ejercicio 1: Creación de una Clase Básica y Manejo de Objetos

Enunciado: crea una clase llamada Empleado para modelar la información de un trabajador en una empresa. La clase debe tener un constructor que inicialice tres atributos de instancia: nombre, salario y puesto. Además, debe tener un método mostrar_informacion() que imprima los detalles del empleado en un formato legible.

Desarrollo Sugerido:

1. Define la clase Empleado con el método constructor __init__(self, nombre, salario, puesto).
2. Dentro del constructor, asigna los parámetros a los atributos de instancia self.nombre, self.salario y self.puesto.
3. Define un método mostrar_informacion(self) que use *f-strings* para imprimir el nombre, salario y puesto del empleado.
4. Crea dos objetos de la clase Empleado con información de tu elección.
5. Llama al método mostrar_informacion() en cada uno de los objetos creados.

Solución Esperada:
Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 1

Ejercicio 1 Unidad 2.

```python
class Empleado:
    def __init__(self, nombre, salario, puesto):
        self.nombre = nombre
        self.salario = salario
        self.puesto = puesto

    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Puesto: {self.puesto}, Salario: ${self.salario:,.2f}")

# Creación de objetos
empleado1 = Empleado("Ana Torres", 75000, "Gerente de Proyectos")
empleado2 = Empleado("Luis Soto", 55000, "Analista de Datos")

# Llamada a métodos
empleado1.mostrar_informacion()
empleado2.mostrar_informacion()
```

Ejercicio 2: Uso de try…except…else…finally

Enunciado: desarrolla un programa que solicite al usuario dos números y realice una división entre ellos. El programa debe manejar de manera robusta las siguientes situaciones:

1. Si el usuario ingresa un valor no numérico (ej. texto), debe capturar el error y mostrar un mensaje amigable.
2. Si el usuario ingresa el número 0 como divisor, debe capturar el error y notificar al usuario.
3. Si la operación se realiza con éxito, el programa debe imprimir el resultado dentro de un bloque else.
4. Finalmente, un bloque finally debe imprimir un mensaje de finalización, sin importar el resultado de la operación.

Universidad Andrés Bello

Desarrollo Sugerido:

1. Define un bloque try.
2. Dentro del try, solicita al usuario dos entradas usando input() y conviértelas a tipo float.
3. Realiza la operación de división.
4. Define un except ValueError para manejar entradas no numéricas.
5. Define un except ZeroDivisionError para manejar la división por cero.
6. Añade un bloque else que se ejecute solo si la división es exitosa y muestre el resultado.
7. Añade un bloque finally que imprima un mensaje final.

Solución Esperada:
Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 2
Ejercicio 2 Unidad 2.

```python
try:
    num1_str = input("Ingrese el dividendo: ")
    num2_str = input("Ingrese el divisor: ")

    num1 = float(num1_str)
    num2 = float(num2_str)

    resultado = num1 / num2

except ValueError:
    print("Error: Ingrese valores numéricos válidos.")
except ZeroDivisionError:
    print("Error: No se puede dividir por cero.")
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")

else:
    print(f"El resultado de la división es: {resultado}")

finally:
    print("Programa de división finalizado.")
```

Universidad Andrés Bello

Ejercicio 3: Herencia en Clases

Enunciado: modifica la clase Empleado del primer ejercicio y crea una jerarquía de herencia. Crea una nueva clase llamada Gerente que herede de Empleado. La clase Gerente debe tener un atributo adicional llamado departamento y un método propio llamado asignar_tareas() que reciba una lista de tareas y las imprima.

Desarrollo Sugerido:

1. Define la clase Empleado como en el Ejercicio 1.
2. Define la clase Gerente que herede de Empleado.
3. En el constructor __init__ de Gerente, llama al constructor de la superclase Empleado usando super().__init__() para inicializar nombre y salario.
4. En el mismo constructor de Gerente, inicializa el nuevo atributo self.departamento.
5. Crea un método asignar_tareas(self, lista_tareas) en la clase Gerente que itere sobre la lista de tareas y las imprima.
6. Crea un objeto de la clase Gerente y llama tanto a un método heredado (mostrar_informacion()) como a su método propio (asignar_tareas()).

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 3

Ejercicio 3 Unidad 2.

```python
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Salario: ${self.salario:,.2f}")

class Gerente(Empleado):
    def __init__(self, nombre, salario, departamento):
        super().__init__(nombre, salario)  # Llama al constructor de la clase padre
        self.departamento = departamento

    def asignar_tareas(self, lista_tareas):
        print(f"\n{self.nombre} ha asignado las siguientes tareas al departamento de {self.departamento}:")
        for i, tarea in enumerate(lista_tareas, 1):
            print(f"{i}. {tarea}")

# Creación de objeto Gerente
gerente1 = Gerente("Carlos Ruiz", 120000, "Desarrollo de Software")

# Usar métodos heredados y propios
gerente1.mostrar_informacion()

tareas_pendientes = ["Planificar sprint", "Revisar código", "Reunión de equipo"]
gerente1.asignar_tareas(tareas_pendientes)
```

Ejercicio 4: Aplicación del Polimorfismo

Enunciado: utiliza las clases Empleado y Gerente del ejercicio anterior y crea una lista que contenga objetos de ambos tipos. Luego, define una función que itere sobre la lista y llame a un método común para cada objeto, demostrando el polimorfismo.

Desarrollo Sugerido:

1. Asegúrate de tener las clases Empleado y Gerente definidas.
2. Crea una lista llamada equipo que contenga una combinación de objetos Empleado y Gerente.

3. Crea una nueva clase, por ejemplo, Desarrollador, que herede de Empleado y sobreescriba el método mostrar_informacion() para que también muestre el lenguaje de programación que utiliza.
4. Añade un objeto Desarrollador a la lista equipo.
5. Crea una función llamada reporte_equipo(lista_personal) que reciba la lista como parámetro.
6. Dentro de la función, itera sobre la lista y llama al método mostrar_informacion() de cada objeto.

Solución Esperada:
Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 4
Ejercicio 4 Unidad 2.

```python
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario
    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Salario: ${self.salario:,.2f}")

class Gerente(Empleado):
    def __init__(self, nombre, salario, departamento):
        super().__init__(nombre, salario)
        self.departamento = departamento
    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Salario: ${self.salario:,.2f}, Puesto: Gerente de {self.departamento}")

# Nueva clase para demostrar polimorfismo
class Desarrollador(Empleado):
    def __init__(self, nombre, salario, lenguaje):
        super().__init__(nombre, salario)
        self.lenguaje = lenguaje
    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Salario: ${self.salario:,.2f}, Rol: Desarrollador ({self.lenguaje})")

# Crear una lista con objetos de diferentes tipos
equipo = [
    Empleado("Ana López", 60000),
    Gerente("Pedro García", 150000, "Marketing"),
    Desarrollador("Sofía Mora", 70000, "Python")
]

# Función polimórfica
def reporte_equipo(lista_personal):
    print("--- Reporte del Equipo ---")
    for persona in lista_personal:
        persona.mostrar_informacion()

# Llamada a la función
reporte_equipo(equipo)
```

Universidad Andrés Bello

Ejercicio 5: Uso de assert para Validación de Clases

Enunciado: modifica la clase Empleado para incluir una validación en el constructor utilizando la sentencia assert. El programa debe verificar que el salario sea un número positivo y que el nombre sea una cadena de texto no vacía. Si alguna de estas condiciones no se cumple, el programa debe lanzar un AssertionError con un mensaje claro.

Desarrollo Sugerido:

1. En el método __init__ de la clase Empleado, antes de asignar los atributos, utiliza la sentencia assert.
2. El primer assert debe verificar que isinstance(nombre, str) y que len(nombre) > 0, con un mensaje de error apropiado.
3. El segundo assert debe verificar que isinstance(salario, (int, float)) y que salario > 0, con un mensaje de error apropiado.
4. Crea un objeto Empleado con datos válidos para verificar el éxito.
5. Usa un bloque try...except para intentar crear un objeto Empleado con datos inválidos (ej. un salario negativo o un nombre vacío) y captura el AssertionError.
6. Imprime el mensaje de error capturado en el except.

Solución Esperada:
Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 5

Ejercicio 5 Unidad 2.

```python
class Empleado:
    def __init__(self, nombre, salario):
        # Validaciones con assert
        assert isinstance(nombre, str) and len(nombre) > 0, "El nombre debe ser una cadena de texto no vacía."
        assert isinstance(salario, (int, float)) and salario > 0, "El salario debe ser un número positivo."

        self.nombre = nombre
        self.salario = salario

    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}, Salario: ${self.salario:,.2f}")

# Escenario exitoso
empleado_valido = Empleado("María Pérez", 80000)
empleado_valido.mostrar_informacion()

# Escenario con error (usando try...except para capturar el AssertionError)
try:
    print("\nIntentando crear un empleado con salario inválido...")
    empleado_invalido = Empleado("David", -5000)
except AssertionError as e:
    print(f"Error capturado: {e}")

try:
    print("\nIntentando crear un empleado con nombre inválido...")
    empleado_invalido_nombre = Empleado("", 50000)
except AssertionError as e:
    print(f"Error capturado: {e}")
```